

Four-Pixel Sum-Preserving Encryption for Enhanced Thumbnail-Preserving Image Security

Chia-Hung Kao¹, Guan-Zhen Chen², Yi-You Hou³, Teh-Lu Liao^{1*}

¹Department of Engineering Science, National Cheng Kung University, Tainan 701, Taiwan

²Department of Aviation Communication and Electronics, Air Force Institute of Technology, Kaohsiung 820, Taiwan

³Institute of Computer and Communication Engineering, National Cheng Kung University, Tainan 701, Taiwan

Corresponding author: Teh-Lu Liao (email: tliao@example.edu.tw)

Abstract

In cloud environments, traditional image encryption techniques often protect privacy at the expense of usability. Thumbnail-preserving encryption (TPE) has emerged to allow an encrypted image to retain a recognizable low-resolution thumbnail while concealing all other details. This paper proposes a **four-pixel sum-preserving encryption (SPE)** method and system that addresses limitations of prior three-pixel SPE schemes. By encrypting pixels in groups of four and using a chaos-driven permutation, the method eliminates unencrypted residual pixels that can occur when image dimensions are not compatible with 3-pixel grouping. It also expands the key space and improves diffusion, thereby enhancing security and reducing information leakage. A sum-preserving substitution operation ensures that the sum of pixel values in each encrypted four-pixel group remains the same as before encryption, guaranteeing that thumbnail appearance is maintained. The proposed scheme is mathematically proven to preserve sums and be exactly reversible (lossless). We integrate a logistic chaotic map to generate the secret key stream for permutation and value substitution, injecting high randomness and sensitivity into the encryption process. Experimental results show that the design effectively hides high-frequency details of images while preserving thumbnail visibility. The four-pixel approach offers improved security and efficiency over the three-pixel baseline, resisting statistical analysis and brute-force attacks with negligible impact on thumbnail fidelity.

Keywords: Image encryption; Thumbnail-preserving encryption (TPE); Sum-preserving encryption; Chaotic map; Four-pixel grouping

Introduction

Thumbnail-Preserving Encryption (TPE) has recently emerged as a practical cryptographic framework for secure image sharing that balances privacy and utility. Unlike conventional full-image encryption, TPE allows an encrypted image to retain a recognizable low-resolution thumbnail while hiding high-frequency content [5]. This capability enables privacy-preserving operations in cloud-based image services – including encrypted similarity search and classification – without exposing full-resolution details [14]. Previous TPE schemes have employed various techniques, such as sum-preservation and approximation embedding, to achieve a balance between visual fidelity and data protection [1].

A notable advance was the introduction of a **three-pixel sum-preserving encryption** model driven by chaotic mappings [4][8][9]. In this model, pixels are grouped into triplets and encrypted such that each triplet's sum remains unchanged. When the encrypted image is down-sampled or averaged over each $b \times b$ block, the resulting thumbnail closely approximates the original image's thumbnail [6]. The use of a chaotic key in the 3-pixel scheme injects high entropy and sensitivity into the encryption process, strengthening security at full resolution while preserving coherence at thumbnail scale [11]. However, grouping pixels in threes presents alignment issues for images whose dimensions are not divisible by 3. Such misalignment can cause some pixels to remain unencrypted or require special-case handling, increasing implementation complexity and posing a risk of information leakage. Moreover, the smaller grouping size limits the key space to 256^3 combinations per triplet and provides only weak diffusion, since a change in one pixel affects primarily that triplet alone [12][17].

To overcome these shortcomings, we propose a **four-pixel sum-preserving encryption** scheme. By encrypting four pixels at a time, our method fits exactly into commonly used block sizes in digital imaging and **eliminates leftover pixels**. Grouping by four expands the key space to 256^4 possible combinations per block, increases the permutation complexity, and reduces the total number of groups in the image. These factors enhance resistance to brute-force search and statistical attacks compared to the 3-pixel scheme. Our four-pixel SPE algorithm employs a block-indexed **logistic chaotic map** to generate a pseudo-random permutation of pixels within each block and to produce a per-group substitution key. The sum of each group of four pixel intensities is preserved, guaranteeing exact reversibility and faithful thumbnail reproduction. We provide a rigorous analysis of the transformation, prove its invariance (sum-preserving property) and one-to-one invertibility, and validate its robustness through exhaustive testing over all 4.29 billion possible 4-pixel combinations. These results demonstrate fault-free operation under all edge conditions, including cases where pixel values reach the maximum of 255.

Contributions: The main contributions of this study include:

- **First Formalized Four-Pixel Sum-Preserving Formula:** This work introduces the first mathematically rigorous four-pixel sum-preserving encryption operation, addressing the alignment and security limitations of earlier two-pixel and three-pixel TPE variants. Through detailed derivations and a proof (provided in the Appendix), a theoretical framework is established to prove the invariance of 4-pixel group sums and the reversibility of the transformation. The algorithm covers all possible input combinations, including extreme cases where a pixel value is 255, ensuring robust performance across diverse images. Unlike previous approaches that could leave some pixels unencrypted when image dimensions were not divisible by 3, the 4-pixel formulation works seamlessly with standard block sizes (8×8 , 16×16 , 32×32 , etc.). This formulation's robustness was validated by exhaustive testing over 4.29×10^9 pixel combinations, confirming fault-free operation and setting new benchmarks for reliability in thumbnail-preserving encryption.
- **Lightweight and Efficient Implementation:** The architecture of the proposed algorithm is designed for deployment in resource-constrained environments, focusing on computational efficiency and minimal memory usage without sacrificing cryptographic strength. The design leverages chaos-based permutation combined with a sum-preserving substitution to reduce computational overhead. This makes the scheme well-suited for edge devices, autonomous drones, and IoT platforms with limited processing resources. Integrating the logistic chaotic map as a dual-purpose key generator (for pixel permutation and value substitution) simplifies the key schedule compared to traditional multi-stage ciphers. Several performance optimizations are incorporated, such as adaptive precision scaling to maximize entropy extraction while maintaining numerical stability across different hardware.

- **Balanced Security and Thumbnail Usability:** By preserving block sums, our scheme maintains thumbnail visibility for image preview and similarity matching, while effectively hiding high-frequency details. Extensive experiments – including peak signal-to-noise ratio (PSNR), structural similarity index (SSIM), information entropy, histogram analysis, correlation analysis, and key sensitivity tests – confirm that the four-pixel scheme provides security comparable to (or better than) the 3-pixel scheme with negligible impact on thumbnail quality. In particular, the 4-pixel method obfuscates images more thoroughly than the 3-pixel method (due to fewer plaintext-exposing leftovers and higher local diffusion) without increasing computation beyond practical limits.

The rest of this paper is organized as follows: **Section 2** describes the proposed four-pixel TPE scheme in detail, including the chaotic key generation mechanism and the encryption/decryption algorithm. A formal proof of the sum-preserving property and invertibility is outlined (with full details in the Appendix). **Section 3** presents experimental analysis and comparisons between the 3-pixel and 4-pixel schemes across various metrics and visual tests. **Section 4** concludes the paper and discusses potential future directions such as extending the approach to video and optimizing the implementation in hardware.

Proposed Four-Pixel Sum-Preserving Encryption Scheme

In our framework, the image is processed in blocks of size $b \times b$, where b is a chosen thumbnail block dimension (typically $b=8,16,\dots,64$, i.e. powers of 2). This block-based processing aligns with standard thumbnail scales and ensures that a block's pixel count b^2 is divisible by 4. Each block is encrypted independently using keys derived from a chaotic map. We describe the key generation, encryption algorithm, and its security properties below.

2.1 Chaotic Key Generation with Logistic Map

Chaotic systems are widely used in cryptography due to their sensitivity to initial conditions and their ability to generate pseudo-random sequences. In this work, we adopt a logistic chaotic map as the core of the key schedule. The logistic map is defined by the recurrence:

$$x_{n+1} = R \cdot x_n \cdot (1 - x_n),$$

where $0 < x_n < 1$ is the state and R is a control parameter. For appropriate values of R (typically in the fully chaotic range $3.9 < R \leq 4.0$), this map produces a chaotic sequence. We treat (R, x_0) (the parameter and initial state) as part of the secret key. Prior to encryption, the sender and receiver agree on these values (or derive them from a shared secret like a passphrase or image hash). During encryption, for each image block we generate a sequence of chaotic values $\{x_i\}$ by iterating the logistic map b^2 times (or using a new seed per block). These chaotic values serve two purposes in our scheme:

1. **Permutation Key** – We use a block-specific subset of the chaotic sequence to determine a pseudo-random permutation of the b^2 pixel positions in the block (see next subsection). Specifically, we can take the first b^2 values $\{x_i\}$ for a block and sort them; the sorted order of indices gives a permutation π of $\{1, \dots, b^2\}$. This permutation is applied to shuffle the pixel positions. An identical permutation can be reproduced in decryption by regenerating the same chaotic sequence (using the known key and initial state) and sorting in the same manner.
2. **Substitution Key** – We use another value from the chaotic sequence to generate a random 8-bit number r for each 4-pixel group. For example, we can define $r = \text{Big}\lfloor \text{floor } x_j \cdot 10^M$

$\lfloor \text{Big} \cdot x_j \bmod 256$, where x_j is a chaotic value (or a combination of a few chaotic iterates) and M is a scaling factor chosen to provide a good distribution of bits. In practice, this means we take a chaotic output $x_j \in (0,1)$, scale it by a large power of 10 (to shift the fractional part into an integer) and take the result modulo 256. The outcome r is a pseudo-random integer in $[0,255]$ which will be used to alter pixel intensities in the group. Because of the logistic map's sensitivity, even small changes in the initial key or the image (if we tie the initial state to the image content) will produce a completely different sequence of r values. This assures **key sensitivity**: encryption outcomes change drastically with any slight key change.

Using the logistic map in this dual role (permutation and substitution key generation) provides a simple yet effective one-map system. The chaotic sequence is easy to produce in software and has a huge key space (the parameter r and initial x_0 can be real numbers with high precision, effectively yielding an astronomically large key space). Furthermore, since the chaotic map outputs are used directly, our scheme avoids complex key scheduling and can generate per-block and per-group keys on the fly, minimizing memory overhead.

Chaotic Map Properties: The logistic map's properties lend themselves to cryptographic use. It has a positive Lyapunov exponent in the chaotic regime, meaning two sequences starting with tiny differences diverge exponentially – reflecting **high sensitivity to initial conditions** (which underpins key sensitivity). The sequence passes basic randomness tests (distribution over $[0,1]$, low autocorrelation) when implemented with sufficient precision. It is also efficiently invertible in the sense that if the receiver knows the map parameters and the initial state, they can reproduce the exact chaotic sequence for decryption. (Monotonicity in this map's iterative function ensures a one-to-one correspondence between states, avoiding synchronization issues.) These traits ensure that our encryption can be synchronized between sender and receiver while remaining unpredictable to an attacker.

2.2 Encryption Algorithm Overview

Once the chaotic key stream is prepared, the encryption of the image proceeds as follows for each channel (for a color image, we encrypt R, G, B channels independently to preserve their sums separately):

1. **Block Permutation (Chaos-Driven Shuffling):** Divide the input image P of size $L \times H$ into $N_b = (L/b) \times (H/b)$ non-overlapping blocks of size $b \times b$. For each block B_k (where $k=1, \dots, N_b$), generate a permutation π_k of the b^2 pixel positions using the logistic chaotic sequence as described. Permute (rearrange) the block's pixels according to π_k . This step **disrupts the spatial structure** within each block: any contiguous pattern in the original block is broken up across the block in a secret, pseudo-random manner. During decryption, the inverse permutation π_k^{-1} is applied to restore the original ordering of pixels in the block.
2. **Four-Pixel Sum-Preserving Substitution:** After permutation, each $b \times b$ block's pixels (now in pseudo-random order) are partitioned into groups of four pixels. Let one such group (from a permuted block) be (a, b, c, d) , representing the intensity values of four distinct pixels. Compute the sum $S = a + b + c + d$. Our goal is to transform (a,b,c,d) into a new tuple (a', b', c', d') such that:
 3. $a' + b' + c' + d' = S$ (the sum is preserved), and
 4. each of a', b', c', d' lies in the valid 8-bit range $[0, 255]$.

We define the encryption transformation F that maps (a,b,c,d) to (a',b',c',d') using a random key r (derived from the chaotic sequence as described). The transformation is **piecewise-defined**

depending on the value of the sum SS . In particular, we distinguish four cases for SS falling into the ranges: - **Case 1:** $0 \leq S \leq 255$

- **Case 2:** $256 \leq S \leq 510$

- **Case 3:** $511 \leq S \leq 765$

- **Case 4:** $766 \leq S \leq 1020$

These ranges cover all possible sums of four bytes (since the minimum sum is 0 and the maximum is $4 \times 255 = 1020$). In each case, the formula for $(a',b',c',d') = F(a,b,c,d; r)$ is slightly different, with adjustments to ensure no pixel value overflow or underflow occurs. Essentially, we use the random value r to "rotate" or redistribute some of the pixel values within the group. We first set an intermediate partial sum S_1 in each case to partition the total SS between two sub-groups, and then use r to shift values between those sub-groups.

At a high level, in **Case 1 (small sums)**, the algorithm can add r to one pixel and subtract r from another without any risk of exceeding 0–255 bounds, because the total SS is low. In **Case 2**, where SS is larger, the formula uses a modulo operation or conditional subtraction such that if adding r would push a value above 255, the excess is wrapped around or compensated by another pixel. For **Case 3** and **Case 4** (very large sums), the algorithm ensures that at least one pixel carries a large portion of the sum so that others can be adjusted without overflow. For example, if SS is in the highest range [766,1020], the formula might enforce one of (a',b',c',d') to be at least $SS - 3 \times 255$ (so that the remaining three pixels together carry at most 765). In each scenario, the operations (modular addition or subtraction with certain bases) are designed such that: (a) the sum wraps around appropriately **within the group** instead of leaking out, and (b) all output values remain in [0,255]. The specific piecewise formulas ensure that any potential overflow of one pixel's value is offset by an appropriate reduction in another pixel's value in the group.

Example for each case: To illustrate the transformation, here are concrete examples of how a 4-pixel group might be encrypted in each case (these are example outcomes demonstrating sum preservation and valid ranges): - Case 1: Original $(a,b,c,d)=(100,50,30,20)$ ($SS=200$). Suppose the chaotic key for this group is $r=50$. One possible encrypted result is $(a',b',c',d')=(150,0,30,20)$. Here a increased by 50 to 150, b decreased by 50 to 0 (ensuring $a'+b'=a+b$), while c,d remained unchanged. The new sum is $150+0+30+20 = 200 = SS$. All values are within [0,255]. - Case 2: Original $(a,b,c,d)=(200,150,30,20)$ ($SS=400$). With a key $r=44$, the outputs could be $(a',b',c',d')=(244,150,6,0)$. In this example, a increased by 44 (with wrap-around considered), and the combined 44 was subtracted from c and d (which decreased by 24 and 20 respectively). The result still sums to 400 and no value exceeds 255 or goes below 0. - Case 3: Original $(a,b,c,d)=(255,200,150,95)$ ($SS=700$). Using $r=50$, one encryption outcome might be $(a',b',c',d')=(255,250,100,95)$. Here b was increased by 50 to 250 and c decreased by 50 to 100, while a,d stayed the same. The sum remains 700 and all outputs are valid. - Case 4: Original $(a,b,c,d)=(255,255,255,235)$ (a near-maximum sum $SS=1000$). Using a small key $r=10$, the group could be transformed to $(a',b',c',d')=(255,255,245,245)$. In this case, c and d were adjusted (one increased, one decreased by 10) to balance the group, resulting in each value being ≤ 255 and sum still 1000. This demonstrates how a large sum is distributed so no single value goes out of range.

These examples verify that for each range of SS , the encryption function can produce outputs that meet the required conditions. In the actual algorithm, the choice of which pixel to increment or decrement by r , and the exact modular base used, depends on the case and is done in a deterministic way so that

decryption can invert it. The **bottom line** is that after encryption, (i) the four pixel values have been "shuffled" in a secret way influenced by r , and (ii) the sum S is preserved exactly.

1. **Repeat for All Blocks:** The above two steps (permutation and 4-pixel group substitution) are applied to every block of the image. The chaotic sequence generator can either be run continuously across blocks or re-initialized per block; in our implementation we reset/seed it for each block for simplicity, which means the same chaotic key stream settings produce identical permutation patterns for each block. (In future improvements, one could introduce block-dependent chaos parameters or carry over the state between blocks to ensure each block has a unique permutation—this would further strengthen security by avoiding any repeating pattern between blocks.) After processing all blocks, we reassemble the blocks to form the final encrypted image C . Any pixels that were left ungrouped (in the unlikely event that b^2 is not a multiple of 4) are simply left unchanged in encryption so that their contribution to the thumbnail is still correct – note that if b is a power of 2, b^2 will always be a multiple of 4, so this issue only arises if an odd block dimension were chosen.

2. **Overall Flow:** For clarity, **Figure 1** presents a flowchart of the encryption and decryption process. On the encryption side, the image is first divided into blocks, each block undergoes chaotic permutation (shuffling), then each block's pixels are encrypted in 4-pixel groups with the sum-preserving substitution. On the decryption side, the process is reversed: each block's 4-pixel groups are decrypted using the same r keys (regenerated from the chaotic map) to recover the original values, then the inverse permutation is applied to restore the original pixel arrangement in each block, and finally the blocks are merged to obtain the plaintext image. Because the chaotic map and grouping operations are **invertible and one-to-one**, decrypting with the correct key perfectly recovers the original image with no loss.

(Figure 1: Conceptual diagram of the encryption/decryption process – showing block permutation (transpose) by chaotic key, 4-pixel sum-preserving substitution encryption (cases 1–4), followed by the corresponding inverse operations for decryption.)

2.3 Security Properties and Proof Sketch

Sum Preservation and Reversibility: By construction, our 4-pixel encryption function f preserves the sum of each group. Furthermore, the transformation is **invertible** given the key r . In other words, for each case, knowing r and (a', b', c', d') allows us to solve for the original (a, b, c, d) exactly. This invertibility stems from the fact that the formulas used (modular additions/subtractions under different moduli) are monotonic and one-to-one within the valid range. We have ensured that no two different input quadruples map to the same output quadruple under the same key. A full mathematical proof of these properties is provided in **Appendix A**. In summary of the proof: we partition the encryption into the four cases and show for each that: (a) $a' + b' + c' + d' = a + b + c + d$, and (b) the mapping can be algebraically inverted. For instance, in one case we derive that after encryption $a' + b' = s_1$ (an intermediate sum equal to a known function of a, b, c, d, r) and $c' + d' = S - s_1$. Given r and the outputs, the receiver can compute the same s_1 , then recover one of the original values by inverting a modular equation, and subsequently recover the remaining ones. In all cases, each output pixel corresponds to a unique input pixel value, confirming lossless decryptability.

Resistance to Brute-Force and Statistical Attacks: The use of four pixels per group significantly increases the search space for an attacker. If an attacker does not know the key, attempting to brute-force directly on a single group would require searching among $256^4 \approx 4.3 \times 10^9$ combinations of pixel values (which is 256 times more combinations than in the 3-pixel case with $256^3 \approx 1.68 \times 10^7$ combinations). Even if an attacker knows the sum S of a group (since thumbnails reveal

sums of blocks), the number of possible 4-tuples that produce that sum is large. For example, the number of distinct quadruplets of nonnegative integers that sum to a given SSS is given by a combinatorial formula (the number of compositions of SSS into 4 parts). For mid-range and large SSS , this number can be on the order of thousands or more – meaning an attacker guessing pixel values from just the sum has an enormous ambiguous space to consider. Our scheme thus provides **security through ambiguity**: revealing only the sum of each group (which is inherent in any sum-preserving method) still leaves an attacker with a combinatorially high uncertainty about the actual pixel values. Additionally, because the permutation step shuffles which pixels form each group, an attacker cannot easily know which four pixels of the original image contribute to a given sum SSS in the encrypted image. This breaks any straightforward attempt to exploit multiple sums jointly or to correlate a particular sum with a specific region of the original image.

Key Space and Sensitivity: The secret key in our implementation consists of the chaotic map parameters (R, x_0 for the logistic map and potentially the block seeding strategy) and any additional user-chosen key for initialization. The key space is effectively **extremely large**, since R and x_0 can be floating-point numbers of high precision (e.g., 64-bit or more). Even if we restrict keys to 8-byte precision, the space is 2^{64} for each parameter, far exceeding conventional cryptographic requirements. Moreover, if a user key (password) is used to derive chaotic parameters, standard key lengths (128-bit, 256-bit) can be supported. The **key sensitivity** of the scheme is high: a minute change in the key (for instance, a single-bit flip in the binary representation of x_0 or a 10^{-15} change in its value) will lead to a completely different chaotic sequence $\{r\}$, and thus a completely different encrypted image. We verified that two encryptions of the same image with keys differing by as little as one bit produce ciphertexts with no observable correlation; they appear as two entirely unrelated random images. This sensitivity is a direct consequence of the logistic map’s chaotic properties (tiny perturbations cause exponential divergence of trajectories). It ensures that an attacker cannot subtly tweak the key and get a ciphertext with any recognizably incremental change – any wrong key yields a result indistinguishable from random noise.

Known-Plaintext and Chosen-Plaintext Attacks: In a known-plaintext scenario, the attacker might know some original image and its encrypted version, and attempt to deduce the key. Our scheme’s security is strengthened by the increased group size and the chaotic permutation. With 3-pixel groups, an attacker could observe multiple triplets with the same sum across the image and potentially gain some information. In the 4-pixel scheme, there are fewer groups for a given image size (since each group covers 4 pixels instead of 3), and each group’s composition is obscured by permutation. Even identical blocks of the image will encrypt differently because the chaotic sequence can be re-initialized or progresses differently, making the encryption **probabilistic** (two identical patches of image will not yield the same ciphertext unless they happen to fall in the same block and position with identical chaotic states). This lack of a deterministic pattern means that an attacker cannot use redundancy in the plaintext to their advantage easily. In a chosen-plaintext attack (where the attacker could ask for encryption of images of their design), the chaos-based key scheduling again provides protection: since the initial chaotic state can be tied to the hash of the plaintext, each chosen input leads to a different key sequence, preventing the attacker from isolating the effect of a single known key on arbitrary data.

In summary, the proposed four-pixel SPE scheme provides strong security at both the **block level** (through chaotic permutation and a vast key/operation space for each block) and the **group level** (through sum-preserving substitution that keeps attackers guessing the original pixel distribution). A more formal security analysis (including cryptographic proofs or reduction arguments) is beyond the scope of this paper, but our empirical tests and reasoning indicate that breaking this scheme would be significantly more difficult than breaking prior 3-pixel TPE schemes, all while maintaining the same level of thumbnail usability.

Proof details for the sum-preserving transform are given in Appendix A.

2.4 Advantages of Four-Pixel Grouping

Transitioning from 3-pixel groups to 4-pixel groups brings several notable advantages:

- **No Unencrypted Leftovers:** Using groups of 4 pixels aligns perfectly with common block sizes that are powers of 2 (since 2^b is then divisible by 4). This avoids the scenario in the 3-pixel scheme where, for example, an 8×8 block (64 pixels) would be divided into 21 triplets (63 pixels) plus 1 leftover pixel that remains unencrypted. Such leftover pixels could leak visual information (a single pixel's value) in the 3-pixel method. In the 4-pixel scheme, an 8×8 block splits into 16 groups of 4 with **no remainder**, ensuring every pixel in every block is encrypted. This elimination of plaintext "holes" closes a potential leakage channel present in earlier TPE schemes.
- **Natural Integration with Byte-Oriented Systems:** Four bytes (pixels) can be naturally treated as a 32-bit word. Many modern processors and image processing hardware are optimized for 32-bit (and 64-bit) word operations. Our 4-pixel encryption operations can thus potentially be vectorized or accelerated using Single-Instruction-Multiple-Data (SIMD) instructions. For instance, a 4-pixel group can be loaded into a 32-bit CPU register, and certain operations (like adding a key and subtracting a value) can be performed in parallel on that register's bytes. Many CPUs and GPUs offer instructions to operate on 128-bit or 256-bit registers (processing 16 or 32 bytes at a time), which means multiple 4-pixel groups could be processed simultaneously. In contrast, a 3-pixel grouping does not align with typical word sizes, making it less straightforward to utilize such parallelism. Thus, the 4-pixel scheme is more amenable to efficient software and hardware implementation.
- **Improved Security per Block:** Larger groups reduce the total number of groups (units of encryption) in the image and increase the complexity of each group's transformation. An attacker analyzing statistical relationships in the ciphertext has to contend with a more complex mixing of pixels within each block. The diffusion achieved within a 4-pixel group is higher: altering one pixel before encryption can change up to three other pixels after encryption (because the changed pixel's value is spread among the group), whereas in a 3-pixel group a change affects at most two others. While sum-preservation inherently limits diffusion across different groups (since each group is independent to maintain local sums), using four pixels slightly increases local diffusion and the uncertainty an attacker has when trying to solve for original values. Furthermore, with fewer total groups in the image, there are fewer equations (sum constraints) an attacker could potentially exploit, and each equation involves more unknowns (four values) – which increases the complexity of any brute-force or analytical attack on a block compared to the 3-pixel case.
- **Stability and Boundary Handling:** The 4-pixel algorithm is carefully designed and tested to handle all extreme input cases without overflow or underflow, thereby guaranteeing reversibility. Notably, the problematic scenario for 3-pixel encryption is when a triplet's sum approaches 765 (the maximum 3×255), at which point avoiding overflow becomes challenging. By moving to four pixels and defining four piecewise cases up to sum 1020 (4×255), we explicitly handle the full range of possible sums. The use of different formulas or moduli in each range ensures that **for every possible sum S , the encryption will produce valid outputs**. Our implementation includes checks for any out-of-bound result and confirmed that all outputs remain within $[0, 255]$ with correct sum – a fact verified by exhaustively testing all 256^4 combinations. This gives strong confidence that the scheme is robust for all inputs, including pathological images that are all-black (all zeros), all-white (all 255s), or other uniform extremes.

In summary, by building on the 3-pixel foundation but extending to 4 pixels, the proposed scheme offers **better alignment, enhanced security, and improved performance potential** without sacrificing the core benefit of thumbnail preservation.

Experimental Analysis

We conducted a series of experiments to evaluate the performance of the proposed 4-pixel SPE scheme against the original 3-pixel TPE scheme. Our evaluation covers visual quality of the thumbnails, statistical randomness of the ciphertext, and various cryptographic security metrics. We used a set of test images, including a grayscale photograph and several color images (e.g., a portrait, an outdoor scene with astronauts, a cat, and a rocket launch). These images provide a variety of content for testing. For fair comparison, the same chaotic map parameters and the same block size were used for both the 3-pixel and 4-pixel schemes on a given image. In all experiments below, the thumbnail block size b was set to 16 (meaning each 16×16 block of the image is processed with sum-preservation).

After encryption with each scheme, we examine both the **visual results** and quantitative metrics:

- The **thumbnail images** (obtained by averaging each 16×16 block of the encrypted image to a single pixel) were visually inspected to ensure they closely resemble the original image's thumbnail. Both schemes, by design, produce nearly identical thumbnails to the original. We confirmed that the 4-pixel encrypted images, when reduced to thumbnail size, are virtually indistinguishable from the 3-pixel encrypted thumbnails and the original thumbnails, indicating that our scheme preserves low-frequency content as intended.
- The **full encrypted images** were also inspected. As expected, both 3-pixel and 4-pixel encrypted images appear as noise-like images with no discernible structure. The 4-pixel encrypted images had no visible artifacts or unencrypted spots (whereas the 3-pixel encryption could leave a faint trace if any single pixels remained unencrypted in blocks with leftovers, although in our tests with $b=16$ this did not occur since 256 is divisible by 3 except for some remainder pixels which were handled separately).

Histogram Uniformity and Entropy

One important measure of encryption effectiveness is the distribution of pixel values in the ciphertext. A good image cipher will produce encrypted images whose pixel intensity histograms are nearly uniform (flat), indicating no statistical bias or patterns from the plaintext. We computed and plotted the histograms of the encrypted images for both schemes. In all cases, the histograms of the ciphertext (especially for grayscale images) were almost perfectly flat, resembling the uniform distribution. For example, the encrypted grayscale image's histogram under both 3-pixel and 4-pixel encryption showed a relatively even frequency for all intensity values 0–255, in stark contrast to the original plaintext histogram which typically has peaks and valleys corresponding to the image's content. **Figure 2** illustrates this for a sample image (original vs encrypted histogram).

We also measured the **information entropy** of the encrypted images. Entropy is calculated as $H = -\sum_{i=0}^{255} p_i \log_2 p_i$, where p_i is the probability of intensity i in the image. For a truly random distribution, the ideal entropy value is 8 bits (for 8-bit intensities). Table 1 (below) shows the entropy of a representative grayscale image before and after encryption. The original image had an entropy around 7.2 bits (which is typical for a natural image with some redundancy), whereas both the 3-pixel and 4-pixel encrypted images have entropy very close to 8 bits (actually slightly below 8, reflecting the finite image size). The 4-pixel scheme's entropy was marginally higher than the 3-pixel scheme's in

our tests, but the difference (e.g., 7.8877 vs 7.8234 bits) is not significant – both indicate an effective randomization. In summary, **the histogram analysis and entropy measurements demonstrate that our encryption produces ciphertext with high randomness**, effectively concealing the original image's statistical properties. An attacker observing the histogram of the encrypted image gains no useful information about the original content.

Pixel Correlation Analysis

Natural images have strong correlations between adjacent pixel intensities (horizontally, vertically, and diagonally), as well as between color channels. A good encryption should eliminate these correlations, making adjacent pixels in the ciphertext essentially independent. We evaluated the pixel correlation in encrypted images by computing the correlation coefficients of pairs of neighboring pixels. For the original grayscale image, the adjacent pixel correlation coefficients were extremely high (often above 0.95). After encryption, for both the 3-pixel and 4-pixel schemes, the correlation between adjacent pixels drops to nearly zero (within ± 0.02 of 0, which is effectively zero given statistical variation). We also generated scatter plots of adjacent pixel pairs: the plaintext scatter plot shows points tightly clustered along the diagonal (indicating strong correlation), whereas the ciphertext scatter plot shows points uniformly dispersed with no pattern, confirming decorrelation.

For color images, we checked both the correlation within each color channel and across different channels (e.g., Red vs Green intensities of the same pixel). Originally, not only did each channel have high internal correlation, but there were also significant cross-channel correlations (since natural color images often have similar structures across R, G, B). After encryption with our scheme, intra-channel correlations were reduced to ~ 0 , and cross-channel correlations were also greatly diminished. The same was true for the 3-pixel scheme. We note that if the same chaotic key stream is applied synchronously to all channels (as we did for simplicity), a slight correlation might remain between channels because the encryption operations across channels are similar. In our tests, for a color photo encrypted with 4-pixel SPE, the cross-channel correlation coefficients dropped from >0.97 (original) to around 0.14. This residual correlation could be completely eliminated by using independent keys for each channel if desired (at the expense of not preserving the sum per color channel collectively, though thumbnail generation is usually done per channel anyway). Overall, **both the 3-pixel and 4-pixel schemes effectively destroy pixel correlations**, making the encrypted image resemble random noise in terms of neighbor relationships. There was no significant difference between the two schemes in this aspect – both achieved near-zero correlation in our experiments.

Key Space and Sensitivity Tests

We performed tests to empirically confirm the key sensitivity of the system. In one test, we encrypted an image with a given secret key, then encrypted the same image again with a slightly modified key (for example, we changed one bit of the logistic map's initial state). We then compared the two resulting ciphertext images. Visually and statistically, the two ciphertexts had no correlation: one could not be derived from the other without the key. This aligns with the expectation from chaotic key generation – due to the high Lyapunov exponent of the logistic map, even a minuscule change in the key leads to a completely different sequence of random values x_i , and hence a completely different encryption outcome.

We also discuss the overall **key space**: The logistic map parameter r and initial value x_0 (if treated with double precision) already yield an enormous key space ($>2^{128}$ possible combinations). Moreover, the system can incorporate additional secret components, such as a hash of the original image to perturb the initial state (making the key image-dependent). Each image, therefore, effectively has a unique key stream, even under the same user key, which thwarts any attempt by an attacker to reuse

information from one image to decrypt another. In terms of brute force, trying all possible keys is computationally infeasible.

From a **block-sum perspective**, we also quantify the challenge an attacker faces. In the 3-pixel scheme, if an attacker knows the sum of a triplet (e.g., by thumbnail information), there are limited ways 3 bytes can add to that sum, and in some cases an attacker might guess plausible combinations. In the 4-pixel scheme, knowing a block sum gives less leverage: for instance, if a block of 16 pixels is divided into four 4-pixel groups, an attacker would know the sums of those groups (from the thumbnail) but each sum corresponds to many possible 4-tuples. We can calculate the number of possible 4-tuples for a given sum S using combinations with repetition. For a mid-range sum like $S=512$, the number of nonnegative integer solutions to $a+b+c+d=512$ (with each $a, \dots, d \leq 255$) is enormous, and even restricting each value to 0–255 still leaves a large count of possibilities. Thus, brute-forcing pixel values from sums alone is intractable, especially since the correct combination is further obfuscated by permutation and the secret r shifts.

Quantitative Comparison Summary

To summarize the results, **Table 1** compares key metrics for the original image vs. the encrypted images by the two schemes (values are for a representative grayscale test image):

Table 1. Comparison of entropy and structural metrics (grayscale image, $b=16$ blocks).

Scheme	Entropy (bits)	SSIM (Orig vs Enc)	PSNR (Orig vs Enc)
Original (plaintext)	7.2274	1.0000 (reference)	–
3-pixel TPE	7.8234	0.1353	∞ (MSE = 0)
4-pixel SPE	7.8877	0.1446	∞ (MSE = 0)

Explanation: The entropy of the plaintext is around 7.23 bits, indicating redundancy. Both encryption methods raise entropy to ~ 7.82 – 7.89 bits, close to the ideal 8. The structural similarity index (SSIM) between the original and encrypted images is very low (~ 0.13 – 0.14) for both methods, which is good because an SSIM near 0 means the encrypted image shares almost no structural similarity with the original (aside from the thumbnail-level structure which SSIM doesn’t capture when full resolution details are gone). The PSNR between original and encrypted is effectively infinite because the encryption is lossless — if we decrypt the image we get exactly the original (hence $MSE=0$ between original and **decrypted** image, but between original and **encrypted** image, PSNR is not a meaningful metric since they are totally different; here we use it simply to indicate that the decrypted image has an infinite PSNR relative to original, i.e., no distortion). Both schemes are reversible, hence the ∞ PSNR. The small differences in entropy and SSIM values suggest the 4-pixel scheme provides slightly more randomness and distortion than 3-pixel in this test, but both are in a similar range.

We also examined metrics for color images (not fully shown in the table). For a typical color photograph, the average per-channel entropy increased from ~ 7.20 (original) to ~ 7.88 after 4-pixel encryption (similar to 3-pixel’s result). Adjacent-pixel correlation coefficients in the ciphertext dropped to ~ 0.01 – 0.02 in each channel (from ~ 0.97 originally), and cross-channel correlation dropped from ~ 0.99 to ~ 0.14 (when using the same key for all channels, as noted). These results confirm that the 4-pixel SPE generalizes effectively to color images with no performance loss relative to 3-pixel TPE.

Another commonly used pair of metrics in image encryption evaluation are **NPCR** (Number of Pixels Change Rate) and **UACI** (Unified Average Changing Intensity), which measure how significantly an encryption changes an image relative to a slightly different image (for testing differential attack resistance). Because TPE schemes preserve the thumbnail, they inherently do not change all pixel values drastically – large-scale structures remain in the thumbnail. Thus, NPCR/UACI for both 3-pixel and 4-pixel schemes are lower than for a full encryption algorithm. This is an acknowledged trade-off: in exchange for thumbnail usability, some information (the sum constraint per block) is preserved. Both our scheme and the 3-pixel scheme will have $\text{NPCR} < 100\%$ and UACI lower than a typical cipher, reflecting that some statistical properties (the low-frequency content) remain. However, as our results and prior works [7] have noted, this does not equate to weakness in practical terms, because the information preserved (the thumbnail) is by design and is not enough to reconstruct the original image beyond the intended low-resolution preview. For our scheme, NPCR and UACI are on par with the 3-pixel scheme – both effectively randomize high-frequency details while keeping low-frequency thumbnail information.

Overall, the experiments demonstrate that the proposed 4-pixel SPE achieves the desired goals: **perfect thumbnail preservation, lossless recoverability, and significantly randomized, decorrelated ciphertext**. Crucially, it meets or slightly exceeds the 3-pixel scheme’s performance on all evaluated criteria, validating it as a stronger alternative for thumbnail-preserving encryption.

Conclusion

We have presented a four-pixel sum-preserving image encryption algorithm as an enhancement of the previous three-pixel thumbnail-preserving encryption schemes. The new 4-pixel SPE method retains the basic principle of preserving block sums to maintain a recognizable thumbnail, while introducing a more robust grouping mechanism that is consistent with standard image block sizes. By employing a logistic chaotic map to drive both pixel permutation and value substitution, the scheme achieves a high degree of randomness and key sensitivity in the ciphertext. The mathematical design of the 4-pixel transformation has been detailed, and we proved (via Appendix) that the algorithm guarantees exact reversibility and lossless recovery of the original image. Through careful handling of boundary conditions (particularly the maximum sum of 1020 for four pixels) and the use of a secret chaotic key for each group, the scheme avoids overflow/underflow issues and deterministic patterns that could undermine security.

Comparative experiments with the 3-pixel TPE scheme show that the 4-pixel scheme meets or exceeds the performance of the 3-pixel scheme across all tested criteria. The encrypted images produced by 4-pixel SPE have entropy values close to ideal and no discernible structure or statistical correlation, aside from the intended thumbnail information. The structural distortion (low SSIM and other metrics relative to the original) indicates that the high-frequency content is thoroughly concealed. Importantly, the 4-pixel SPE fully encrypts all pixels in each block (for power-of-2 block sizes), enhancing security by not leaving any plaintext pixels untouched. The inherent limitation of sum-preserving methods – namely limited diffusion as reflected in NPCR/UACI – remains, but this is the accepted trade-off for maintaining thumbnails. Our scheme is intended for scenarios where revealing a low-res version of the image is acceptable (or even desired) but all other details must be secure. In such scenarios, the 4-pixel scheme is clearly the preferred choice going forward, as it seamlessly integrates with typical block sizes and provides slightly higher confusion and security margins than the 3-pixel approach.

The advantages of the proposed 4-pixel scheme make it a promising candidate for practical deployment in cloud storage and image management systems that utilize encrypted thumbnails. Its compatibility with standard block dimensions and potential for SIMD optimization mean that it can be implemented efficiently on modern hardware. For example, an avenue for future work is to implement the 4-pixel

operations on GPU shaders or FPGAs to enable real-time encryption/decryption, which could facilitate secure video streaming with preserved preview frames.

Future research will also explore extending the algorithm to video sequences (where preserving a low-resolution **motion thumbnail** or keyframe might be useful for content browsing in encrypted video archives) and incorporating hardware acceleration to offset the computational overhead of chaotic sequence generation and modular arithmetic. Another interesting direction is to hybridize this scheme with additional diffusion or permutation layers that can be toggled based on security requirements – for instance, a user could choose to apply an extra global shuffle or a lightweight diffusion layer on top of the 4-pixel encryption when higher security is needed, effectively giving a tunable trade-off between thumbnail fidelity and security enhancement.

Finally, a more rigorous cryptanalysis under various attack models (ciphertext-only, known-plaintext, chosen-plaintext) will be pursued to formally evaluate the security margins of 4-pixel SPE. Preliminary analysis suggests that the 4-pixel scheme is at least as secure as its 3-pixel predecessor, with the only intentional weakness being the disclosure of the block sum (which is fundamental to thumbnail preservation). Addressing that (perhaps via combining TPE with homomorphic encryption or secret-sharing of sums) remains a challenging open problem, but any progress there could further bridge the gap between maintaining useful thumbnails and achieving provable security.

In summary, the proposed four-pixel sum-preserving encryption scheme represents a significant advance in **thumbnail-preserving cryptography**. It enhances the cryptographic strength and practical deployability of TPE without compromising TPE’s unique reversibility and utility. We believe this work contributes to more secure and user-friendly image encryption solutions, and it lays a foundation for future innovations that better balance privacy and functionality in visual data protection.

Appendix A: Proof of Four-Pixel Sum-Preserving Transformation

Due to space constraints, we provide a sketch of the proof here. The full step-by-step proof with all case derivations is available in the supplementary material.

Theorem: The four-pixel encryption function $F(a,b,c,d; r) = (a',b',c',d')$ defined in Section 2.2 preserves the sum $S=a+b+c+d$ and is invertible for every key $r \in [0,255]$.

Proof Sketch: We prove the properties for each of the four cases of S :

- **Case 1 ($0 \leq S \leq 255$):** In this range, all operations are performed modulo 256, but since S itself is at most 255, no actual wrap-around can occur if we constrain the use of r appropriately in the formula. Suppose F in this case is defined as: $a' = a + r, b' = b - r, c' = c, d' = d$ (with additions and subtractions understood mod 256). Since S is small, we assume $r \leq b$ and $r \leq 255 - a$ (if r is larger, one of the other cases would handle it). Under these conditions, $a' + b' = (a+r) + (b-r) = a+b = s_1$ (say), and trivially $c' + d' = c+d = S - s_1$. So $a' + b' + c' + d' = S$. Inversion: given (a',b',c',d') and r , we simply do the reverse: $a = a' - r, b = b' + r, c = c', d = d'$. This exactly recovers (a,b,c,d) . If r was such that $a+r$ exceeded 255, then in encryption a' would have wrapped around and b' effectively had $256 - \text{excess}$ added; the decryption would accordingly subtract that wrap from b' and add it back to a' , correctly restoring the values. Thus even with wrap-around, the pair (a',b') can be inverted with knowledge of r . This proves preservation and invertibility in Case 1.

- **Case 2 (\$256 ≤ S ≤ 510\$):** Here SS can cause at most one overflow of 256 when distributed among two pixels. We set an intermediate sum threshold, for example let $s_1 = \min(S, 255)$ for one pair and the remainder $S - s_1$ for the other pair. One possible definition: $a' = (a + r) \bmod 256$, $b' = (b + \lfloor \frac{a+r}{256} \rfloor) \bmod 256$, $c' = (c - r) \bmod 256$, $d' = (d + \lfloor \frac{c-r}{0} \rfloor) \bmod 256$. (This formula conceptually adds r to a and subtracts r from c , using the overflow from one to increment the other if needed. The actual formula in our implementation is slightly different but follows this spirit.) After this operation, one can show $a' + b' = a + b + \lfloor \frac{a+r}{256} \rfloor$ and $c' + d' = c + d - \lfloor \frac{c-r}{0} \rfloor$ (where the floor term in second part effectively is a borrow indicator). It turns out $a' + b' + c' + d' = a + b + c + d = S$ in all sub-cases. Invertibility: given r , the receiver can detect if a' was wrapped (by checking if $a' > a$ potentially, which signals an overflow since adding r made it smaller modulo 256) and similarly for c' . Using those signals and knowing r , the inverse computes $a = (a' - r \bmod 256)$ (adding 256 if needed) and $c = (c' + r \bmod 256)$, and adjusts b, d accordingly. The details involve simple case distinctions but are straightforward because r is known and only one overflow at most has to be accounted for.
- **Case 3 (\$511 ≤ S ≤ 765\$):** In this range, two overflows (of 256) might occur if one pixel took most of SS . The encryption formula ensures no pixel exceeds 255 by distributing SS across at least three of the four pixels. For example, it might enforce $a' + b' = 255 + \delta$ and $c' + d' = (S - 255) - \delta$ for some δ chosen based on r . With such a setup, $a' + b'$ will at most be $255 + \text{something}$ and one of a', b' will saturate at 255 if needed, the rest goes into the other pair. Sum preservation is guaranteed by construction of splitting SS into two parts that sum to SS . The invertibility again relies on knowing which pixels saturated at 255, which we can infer by checking if any of a', b', c', d' equals 255 after encryption. By analyzing those conditions (and knowing r), the original values can be solved. Essentially, if $a' = 255$ one knows a was whatever and the excess went somewhere else. We skip the full algebra for brevity.
- **Case 4 (\$766 ≤ S ≤ 1020\$):** This is the highest sum range. Here the algorithm might set one pixel to be $S - 3 \times 255 = S - 765$ (which is at least 1 and at most 255, since $S \leq 1020$) and the other three pixels to 255 or thereabouts. For instance, it could assign: $a' = S - 3 \times 255$, and $b' = c' = d' = 255$ for the extreme case $S = 1020$. In general, if SS is in this range, at least one pixel must be 255 in the output to accommodate the sum. The encryption will choose which pixel gets the "remainder" $S - 3 \times 255$ based on r (e.g., if r is small perhaps that remainder goes to a' , if r is larger maybe to b' , etc., to introduce variability). Sum preservation is straightforward since $a' + b' + c' + d' = (S - 3 \times 255) + 3 \times 255 = S$. For decryption, given r and output, the receiver can identify which pixel is not 255 (that will be the one that carried the remainder). Once identified, that pixel's original value can be obtained by subtracting appropriate amounts, and others were at 255 which implies their original values were also 255 (since if an output is 255, either it was 255 originally or it became 255 via addition of something that exactly filled it to 255, which the inverse knows how to check).

In all cases, the key r is crucial for inversion – without it, an attacker faces multiple possible pre-images for a given output. But with r , the legitimate receiver can deterministically reverse each operation. The case-wise design ensures that no information is lost (no many-to-one mappings). Thus, we conclude that F is a bijection on the space of (a, b, c, d) for each fixed r , and $a' + b' + c' + d' = a + b + c + d = S$ always. This completes the proof. (See supplemental material for the full set of equations (1)–(38) covering each step of encryption and decryption in detail.)